

ANSIBLE INTERVIEW QUESTIONS

Quick Reference Notes for DevOps Interview Prep

BASICS (MUST KNOW)

1 What is Ansible and why is it used in DevOps?

Ansible is an open-source, agentless IT automation platform used to automate configuration management, application deployment, cloud provisioning, and task orchestration.

Why it's used:

- No agent installation required (uses SSH)
- Simple YAML syntax (human-readable)
- Idempotent operations (safe to run multiple times)
- Push-based architecture (more secure)
- Works across multiple platforms (Linux, Windows, cloud)

2 Explain Ansible Architecture

Components:

1. **Control Node** - Machine where Ansible is installed (master)
2. **Managed Nodes** - Target machines to be automated
3. **Inventory** - File listing hosts and groups
4. **Modules** - Pre-built scripts for specific tasks
5. **Playbooks** - YAML files containing automation logic
6. **Plugins** - Extensions for Ansible functionality

Flow:

User → Control Node → SSH → Managed Node → Execute Module → Return Result

3 What is an Ansible Playbook? How is it structured?

Definition: YAML file defining what tasks should run on which hosts.

Basic Structure:

```
---
- name: Description of play
  hosts: target_group
  become: yes/no
  vars:
    variable1: value1
  tasks:
    - name: Task 1 description
      module_name:
        param1: value1
        param2: value2
    - name: Task 2
      module_name:
        option: value
  handlers:
    - name: Restart service
      service:
        name: nginx
        state: restarted
```

Key Elements:

- **hosts** : Target machines from inventory
- **tasks** : Steps to execute sequentially
- **vars** : Variables for this play
- **handlers** : Tasks triggered by notifications
- **become** : Privilege escalation (sudo)

4 What is an Inventory in Ansible?

Definition: Collection of managed hosts organized into groups.

Types:

1. **Static Inventory** - INI or YAML file with fixed host list

2. Dynamic Inventory - Generated from cloud APIs (AWS, Azure, GCP)

Example (INI format):

```
[webservers]
web1.example.com
web2.example.com

[databases]
db1.example.com
db2.example.com

[all:vars]
ansible_user=admin
ansible_ssh_private_key_file=~/.ssh/id_rsa
```

Example (YAML format):

```
all:
  children:
    webservers:
      hosts:
        web1.example.com:
        web2.example.com:
    databases:
      hosts:
        db1.example.com:
```

5 What are Ansible Modules? Name commonly used ones

Definition: Reusable scripts that perform specific tasks.

Common Modules:

- **System:** `user`, `group`, `file`, `service`, `systemd`
- **Package:** `apt`, `yum`, `pip`, `npm`
- **File:** `copy`, `template`, `lineinfile`, `stat`
- **Command:** `shell`, `command`, `script`
- **Network:** `uri`, `get_url`, `wait_for`
- **Cloud:** `ec2`, `azure_vm`, `gcp_compute_instance`
- **Database:** `mysql_user`, `postgresql_user`

6 What is Idempotency in Ansible? Why important?

Definition: Running the same playbook multiple times produces the same result.

Example:

```
- name: Install nginx
  apt:
    name: nginx
    state: present # Idempotent - only installs if not present
```

Why Important:

- Safe to re-run playbooks without side effects
- Predictable state management
- Ideal for configuration drift correction
- Enables reliable automation

7 How do you install Ansible on Linux?

Ubuntu/Debian:

```
sudo apt update
sudo apt install -y ansible
```

RedHat/CentOS:

```
sudo yum install -y ansible
```

Via pip:

```
pip install ansible
```

Verify:

```
ansible --version
```

8 Difference between Ansible, Puppet, Chef vs Terraform

Feature	Ansible	Puppet	Chef	Terraform
---------	---------	--------	------	-----------

Type	Config Mgmt	Config Mgmt	Config Mgmt	Infrastructure as Code
Agent	Agentless	Agent-based	Agent-based	Agent-less
Language	YAML	Ruby	Ruby	HCL
Push/Pull	Push	Pull	Pull	Push
Cloud Focus	Low	Low	Low	HIGH
Learning Curve	Easy	Hard	Hard	Moderate

When to use what:

- **Ansible:** Configuration management, simple deployments
- **Terraform:** Infrastructure provisioning (VMs, networks)
- **Puppet/Chef:** Large-scale enterprise automation

9 What language are Ansible playbooks written in?

Answer: YAML (YAML Ain't Markup Language)

- Simple, readable syntax
- Uses indentation for structure
- Human-friendly format
- No complex coding needed

10 How do you run a playbook from command line?

```
# Basic run
ansible-playbook playbook.yml

# Run on specific inventory
ansible-playbook -i inventory.ini playbook.yml

# Run on specific host
ansible-playbook -i inventory.ini -l hostname playbook.yml

# Run with extra variables
ansible-playbook playbook.yml -e "variable=value"

# Verbose output
ansible-playbook -vvv playbook.yml
```

```
# Dry-run (check mode)
ansible-playbook --check playbook.yml

# Become sudo
ansible-playbook -b playbook.yml
```

PLAYBOOKS, ROLES, VARIABLES

11 Explain playbook components: hosts, tasks, handlers, vars

- **hosts:** Target group/machine
- **tasks:** Steps executed sequentially
- **vars:** Variables for the play
- **handlers:** Tasks triggered by `notify`

Example:

```
- name: Configure web server
hosts: webservers
vars:
  nginx_port: 80
tasks:
  - name: Install nginx
    apt:
      name: nginx
      state: present
      notify: Restart nginx
handlers:
  - name: Restart nginx
    service:
      name: nginx
      state: restarted
```

12 What are handlers in Ansible? When to use them?

Definition: Tasks executed only if notified by other tasks.

Use case: Restart services only when config changes

```
- name: Update nginx config
copy:
  src: nginx.conf
  dest: /etc/nginx/nginx.conf
  notify: Restart nginx

handlers:
  - name: Restart nginx
    service:
```

```
name: nginx
state: restarted
```

13 What are Ansible roles? How do they promote modularity?

Definition: Organized collection of tasks, handlers, vars, templates, and files.

Benefits:

- Reusable across playbooks
- Organized structure
- Share via Ansible Galaxy
- Easier maintenance

Directory Structure:

```
my_role/ ├── tasks/ | └─ main.yml ├── handlers/ | └─ main.yml ├── vars/ | └─
main.yml ├── templates/ | └─ config.j2 ├── files/ | └─ data.txt └─ README.md
```

14 What is the typical directory structure of a role?

```
role_name/ ├── defaults/ | └─ main.yml # Default variables ├── files/ | └─
config.conf # Static files ├── handlers/ | └─ main.yml # Handlers/service
restart ├── meta/ | └─ main.yml # Role dependencies ├── tasks/ | └─ main.yml #
Main tasks ├── templates/ | └─ config.j2 # Jinja2 templates ├── tests/ | └─
test.yml # Test playbook ├── vars/ | └─ main.yml # Variables └─ README.md #
Documentation
```

15 What are the different types of variables and their precedence?

Precedence (highest to lowest):

1. Command line (-e flag)
2. Task vars
3. Block vars
4. Play vars
5. Role vars
6. Host vars
7. Group vars

8. Inventory vars

9. Role defaults

Types:

- **Facts** - Gathered from `setup` module
 - **Registered** - Captured from task output
 - **Play vars** - Defined in playbook
 - **Host/Group vars** - From inventory files
-

16 How do you use Jinja2 templates in Ansible?

Template Module: Process Jinja2 templates with variables

Example template (nginx.conf.j2):

```
server {
    listen {{ nginx_port }};
    server_name {{ server_name }};

    location / {
        proxy_pass http://{{ backend_server }};
    }
}
```

Playbook:

```
- name: Configure nginx
  template:
    src: nginx.conf.j2
    dest: /etc/nginx/sites-available/default
  vars:
    nginx_port: 80
    server_name: example.com
    backend_server: 127.0.0.1:3000
```

17 What are loops and conditionals in Ansible? Give examples.

Loops - `loop` keyword:

```
- name: Create multiple users
  user:
    name: "{{ item }}"
    state: present
```

```
loop:
  - john
  - jane
  - bob

- name: Install packages
  apt:
    name: "{{ item }}"
  loop:
    - nginx
    - git
    - curl
```

Conditionals - **when** keyword:

```
- name: Install nginx on Ubuntu
  apt:
    name: nginx
  when: ansible_os_family == "Debian"

- name: Restart if changed
  service:
    name: nginx
    state: restarted
  when: config_changed.changed
```

18 Difference between `import_tasks` and `include_tasks`

Feature	<code>import_tasks</code>	<code>include_tasks</code>
Static/Dynamic	Static (processed at parse time)	Dynamic (at execution)
Loops	Not supported	Supported
Conditionals	Limited	Supported with conditions
Use case	Reuse fixed task sets	Flexible, dynamic execution

AD-HOC, INVENTORY, CONNECTIONS

19 What is an ad hoc command? Give examples.

Definition: One-off Ansible commands executed directly (not in playbook).

```
# Ping all hosts
ansible all -m ping

# Run shell command
ansible webserver -m shell -a "uptime"

# Install package
ansible dbserver -m apt -a "name=mysql-server state=present" -b

# Gather facts
ansible all -m setup

# Copy file
ansible webserver -m copy -a "src=/etc/config dest=/etc/config"

# Restart service
ansible webserver -m service -a "name=nginx state=restarted" -b
```

20 How do you target specific hosts/groups from CLI?

```
# All hosts
ansible all -m ping

# Specific group
ansible webserver -m ping

# Specific host
ansible web1.example.com -m ping

# Multiple groups
ansible webserver,databases -m ping

# Exclude group
ansible all:!databases -m ping

# Range
ansible web[0:2] -m ping
```

```
# Wildcard
ansible web*.example.com -m ping
```

21 How do you set up SSH access for Ansible?

Key-based (Recommended):

```
# Generate SSH key on control node
ssh-keygen -t rsa -b 4096

# Copy public key to managed nodes
ssh-copy-id -i ~/.ssh/id_rsa.pub user@managed_host

# Verify connectivity
ansible all -m ping
```

Password-based:

```
# Install sshpass
sudo apt install sshpass

# Use in inventory
[webservers]
web1 ansible_user=admin ansible_password=pass123
```

In ansible.cfg:

```
[defaults]
private_key_file = ~/.ssh/id_rsa
remote_user = ubuntu
```

22 What is ansible.cfg and common settings?

Location priority:

1. ANSIBLE_CONFIG environment variable
2. ./ansible.cfg (current directory)
3. ~/.ansible.cfg (home directory)
4. /etc/ansible/ansible.cfg (system)

Common Settings:

```
[defaults]
inventory = /etc/ansible/hosts
private_key_file = ~/.ssh/id_rsa
remote_user = ubuntu
host_key_checking = False
forks = 5
timeout = 10

[privilege_escalation]
become = True
become_method = sudo
become_user = root
become_ask_pass = False
```

23 How do you work with dynamic inventory?

AWS Example:

```
# Install boto3
pip install boto3

# Use AWS plugin (inventory.aws_ec2.yml)
plugin: aws_ec2
regions:
  - us-east-1
keyed_groups:
  - key: tags
    parent_group: tag
```

Custom script (returns JSON):

```
#!/usr/bin/env python3
import json
inventory = {
    "all": {
        "hosts": ["web1", "web2"],
        "vars": {"ansible_user": "ubuntu"}
    }
}
print(json.dumps(inventory))
```

Use it:

```
ansible-playbook -i inventory.py playbook.yml
```

SECURITY, VAULT, SECRETS

24 What is Ansible Vault and when to use it?

Definition: Encrypts sensitive data (passwords, keys) in files.

When to use:

- Store passwords, API keys
- Protect sensitive variables
- Share playbooks safely

Commands:

```
# Create encrypted file
ansible-vault create secrets.yml

# Edit encrypted file
ansible-vault edit secrets.yml

# Encrypt existing file
ansible-vault encrypt playbook.yml

# Decrypt file
ansible-vault decrypt secrets.yml

# Run playbook with vault
ansible-playbook playbook.yml --ask-vault-pass
# Or from file
ansible-playbook playbook.yml --vault-password-file ~/.vault_pass
```

25 How do you create, edit, and use encrypted variables?

Create encrypted vars file:

```
ansible-vault create secret_vars.yml
```

Content (secret_vars.yml):

```
db_password: supersecret123
api_key: abc123xyz789
```

Use in playbook:

```
- hosts: all
  vars_files:
    - secret_vars.yml
  tasks:
    - name: Connect to database
      mysql_user:
        name: admin
        password: "{{ db_password }}"
```

Run with password prompt:

```
ansible-playbook playbook.yml --ask-vault-pass
```

26 How do you manage secrets safely in playbooks?

Best Practices:

1. Use Ansible Vault for sensitive data
2. Store vault password in secure location
3. Never commit passwords to git
4. Use `.gitignore` for secret files
5. Use external secret management (HashiCorp Vault, AWS Secrets Manager)

Example with external vault:

```
- name: Get secret from AWS Secrets Manager
  aws_secret:
    name: my-database-password
    register: db_secret

- name: Use the secret
  mysql_user:
    password: "{{ db_secret.secret }}"
```

ERROR HANDLING, DEBUGGING, PERFORMANCE

27 How do you debug a failing task?

Debug Module:

```
- name: Print variable
  debug:
    var: my_variable

- name: Print message
  debug:
    msg: "Value is {{ variable }}"

- name: Debug registered variable
  debug:
    var: command_result
```

Verbose Flags:

```
# Verbose
ansible-playbook -v playbook.yml

# More verbose
ansible-playbook -vv playbook.yml

# Most verbose
ansible-playbook -vvv playbook.yml

# Enable callback plugin
export ANSIBLE_STDOUT_CALLBACK=debug
```

28 What does --check mode do?

Definition: Dry-run mode - shows what would happen without making changes.

```
# Dry-run playbook
ansible-playbook --check playbook.yml

# Dry-run with verbose
ansible-playbook --check -vv playbook.yml
```



```
# Check + diff (show what would change)
ansible-playbook --check --diff playbook.yml
```

29 What is ignore_errors and when to use?

Definition: Continues playbook even if task fails.

```
- name: Run command that may fail
  shell: /opt/script.sh
  ignore_errors: yes

- name: Continue execution
  debug:
    msg: "This will execute even if above failed"
```

When to use:

- Non-critical tasks
- Tasks with expected failures
- When you want to log errors and continue

30 How do you speed up Ansible execution?

Optimization Techniques:

1. Increase forks (parallel execution):

```
[defaults]
forks = 20  # Default is 5
```

2. Enable fact caching:

```
[defaults]
gathering = smart
fact_caching = jsonfile
fact_caching_connection = /tmp/ansible_facts
fact_caching_timeout = 86400
```

3. Use async for long tasks:

```
- name: Long running task
  shell: /opt/long_script.sh
```

```
async: 300 # Timeout 300 seconds
poll: 10   # Check every 10 seconds
```

4. Disable unnecessary gathering:

```
- hosts: all
  gather_facts: no # Skip if not needed
```

31 What are Ansible facts and how to use setup?

Facts: Variables automatically gathered about managed nodes.

Common facts:

- `ansible_os_family` (Debian, RedHat)
- `ansible_distribution` (Ubuntu, CentOS)
- `ansible_python_version`
- `ansible_processor_cores`
- `ansible_memtotal_mb`

Gather facts:

```
# Gather facts on all hosts
ansible all -m setup

# Filter facts
ansible all -m setup -a "filter=ansible_os_family"
```

Use in playbook:

```
- name: Install package based on OS
  apt:
    name: nginx
  when: ansible_os_family == "Debian"
```

ANSIBLE TOWER / AWX & ECOSYSTEM

32 What is Ansible Tower (or AWX)? Key features.

Definition: Web UI and REST API for Ansible at enterprise scale.

Key Features:

- Web-based dashboard
- Role-based access control (RBAC)
- Job scheduling
- Credential management
- Inventory management
- Workflow execution
- Audit logging
- Real-time job output

Use Cases:

- Multi-team automation
- Compliance and governance
- Self-service automation
- Integration with external tools

33 How do you schedule jobs or control RBAC with Tower?

Job Scheduling:

- Create schedule in Tower UI
- Select playbook and inventory
- Set frequency (daily, weekly, monthly)
- Specify execution time

RBAC:

- Users, Teams, Organizations
- Permissions (admin, read, execute)
- Credential access control
- Template access levels

34 What is Ansible Galaxy? How to use roles?

Definition: Community hub for sharing Ansible roles and collections.

Usage:

```
# Search roles
ansible-galaxy search nginx

# Install role
ansible-galaxy install geerlingguy.nginx

# Use in playbook
- hosts: webservers
  roles:
    - geerlingguy.nginx

# List installed roles
ansible-galaxy list
```

SCENARIO-BASED / PRACTICAL QUESTIONS

35 Design Ansible solution for web server (Nginx) on multiple nodes

Playbook structure:

```
---
- name: Configure Nginx web servers
  hosts: webservers
  become: yes
  vars:
    nginx_user: www-data
    nginx_port: 80

  roles:
    - role: geerlingguy.nginx
      vars:
        nginx_vhosts:
          - name: example.com
            upstream_servers:
              - "127.0.0.1:3000"

  tasks:
    - name: Ensure nginx is started
      service:
        name: nginx
        state: started
        enabled: yes
```

36 How to perform rolling updates with zero downtime?

```
---
- name: Rolling update with zero downtime
  hosts: webservers
  serial: 1 # Update one server at a time

  tasks:
    - name: Remove from load balancer
      uri:
        url: "http://lb.example.com/remove/{{ inventory_hostname }}"
        method: POST

    - name: Stop application
      service:
        name: app
        state: stopped
```

```
- name: Deploy new version
  git:
    repo: https://github.com/example/app.git
    dest: /opt/app
    version: v2.0

- name: Start application
  service:
    name: app
    state: started

- name: Health check
  uri:
    url: "http://{{ inventory_hostname }}:8000/health"
    register: health
    until: health.status == 200
    retries: 5

- name: Add back to load balancer
  uri:
    url: "http://lb.example.com/add/{{ inventory_hostname }}"
    method: POST
```

37 How to enforce security baselines/compliance?

```
---
- name: Security baseline compliance
  hosts: all
  become: yes

  tasks:
    - name: Disable root SSH login
      lineinfile:
        path: /etc/ssh/sshd_config
        regexp: '^#?PermitRootLogin'
        line: 'PermitRootLogin no'
        notify: Restart SSH

    - name: Set SSH port
      lineinfile:
        path: /etc/ssh/sshd_config
        regexp: '^#?Port'
        line: 'Port 2222'
        notify: Restart SSH

    - name: Update all packages
      apt:
        update_cache: yes
        upgrade: dist

    - name: Install security tools
```

```
apt:
  name: "{{ item }}"
  state: present
loop:
  - fail2ban
  - aide
  - auditd

- name: Enable firewall
  ufw:
    state: enabled
    policy: deny
    direction: incoming

- name: Allow SSH
  ufw:
    rule: allow
    port: '2222'
    proto: tcp

handlers:
  - name: Restart SSH
    service:
      name: ssh
      state: restarted
```

38 How to integrate Ansible with Jenkins CI/CD?

```
# Jenkins pipeline (Groovy)
pipeline {
  agent any

  stages {
    stage('Checkout') {
      steps {
        git 'https://github.com/example/repo.git'
      }
    }

    stage('Run Playbook') {
      steps {
        sh 'ansible-playbook -i inventory.ini deploy.yml --vault-password-file=vault.txt'
      }
    }

    stage('Health Check') {
      steps {
        sh 'ansible-playbook health_check.yml'
      }
    }
  }
}
```

```
}  
}
```

39 How to structure large Ansible project with roles?

```
project/ ├── inventory/ | ├── production | ├── staging | └── development |  
roles/   | ├── common/  | ├── webserver/  | ├── database/  | └── monitoring/ |  
playbooks/ | ├── site.yml | ├── deploy.yml | └── maintenance.yml |── group_vars/  
| ├── all.yml | ├── webservers.yml | └── databases.yml |── host_vars/ | └──  
web1.example.com.yml |── library/ | └── custom_module.py |── filter_plugins/ |  
└── custom_filter.py |── ansible.cfg └── README.md
```


QUICK REFERENCE TIPS

Before Interview:

- Practice writing playbooks
- Know your projects deeply
- Understand Ansible architecture
- Be ready for hands-on coding
- Prepare real-world scenarios

Common Mistakes to Avoid:

- Forgetting `become: yes` for sudo tasks
- Not using idempotent modules
- Poor variable naming
- Missing error handling
- Not using templates for config files

Speed Tips:

- Use roles for reusability
- Cache facts for performance
- Use async for parallel execution
- Optimize inventory management

Additional Resources:

- Official Docs: <https://docs.ansible.com>
- Ansible Galaxy: <https://galaxy.ansible.com>
- Red Hat Training: <https://www.redhat.com/en/services/training>
- Community: <https://www.ansible.com/community>

Last Updated: November 28, 2025

Prepared for: DevOps/SRE Interview Candidates